Lab 3: Hàm, đệ quy, con trỏ và cấp phát động (Functions, Pointers & Dynamic Memory)

Hướng dẫn thực hành Lập trình hệ thống

Trường Đại học Công nghệ - ĐHQGHN

Ngày 3 tháng 10 năm 2026

Mục lục

1. Mở đầu: từ Lab 2 sang Lab 3
2. Hàm
3. Call stack và đệ quy
4. Con trỏ
5. Con trỏ, mảng, chuỗi và tìm kiếm nhị phân
6. Cấp phát bộ nhớ động
7. Lỗi bộ nhớ và Valgrind
8. Danh sách liên kết đơn
9. Bài tập thực hành
10. Tổng kết

Nội dung

1. Mở đầu: từ Lab 2 sang Lab 3
2. Hàm
3. Call stack và đệ quy
4. Con trỏ
5. Con trỏ, mảng, chuỗi và tìm kiếm nhị phân
6. Cấp phát bộ nhớ động
7. Lỗi bộ nhớ và Valgrind
8. Danh sách liên kết đơn
9. Bài tập thực hành
10. Tổng kết

Mục tiêu buổi thực hành

Sau buổi hôm nay, sinh viên phải làm được

1. **Tổ chức chương trình thành hàm:** khai báo, định nghĩa, gọi hàm; phân biệt *phạm vi* và *thời gian sống* của biến.
2. **Viết đệ quy đúng:** có điểm dừng, có bước tiến; giải thích call stack, stack overflow và quan sát chúng bằng gdb.
3. **Dùng con trỏ:** truyền địa chỉ (`swap`, tham số đầu ra), duyệt mảng và chuỗi bằng con trỏ, tránh bẫy `sizeof`.
4. **Cấp phát động đúng mẫu:** `malloc`/`calloc`/`realloc`/`free`, và *chứng minh* chương trình không rò rỉ bằng Valgrind.

Sản phẩm cần nộp

20 chương trình `bai01.c` ... `bai20.c` (5 hàm, 5 đệ quy, 5 con trỏ, 5 cấp phát động), biên dịch `-Wall -Wextra` **sạch cảnh báo**; Bài 16–20 chạy `valgrind` **sạch lỗi, không rò rỉ**.

Trả lời câu hỏi của tuần trước: vì sao scanf cần &age?

```c
int age;
/* dua cho scanf DIA CHI cua age */
scanf("%d", &age);
```

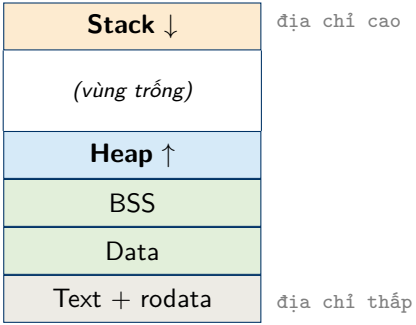
C luôn **truyền tham trị**: hàm nhận *bản sao* đối số. Bản sao *giá trị* của age (đang là rác) vô dụng với scanf; bản sao *địa chỉ* thì cho scanf biết phải **ghi vào đâu**. Biến giữ địa chỉ gọi là **con trỏ**.

Học ở Lab 3	Gặp lại ở các tuần sau
Tham số đầu ra `int *`	`wait(&status)`, `pipe(fd)`
Bộ đệm `buf` + độ dài `n`	`read(fd, buf, n)`
`malloc`/`free`	Đọc file chưa biết kích thước
Mã lỗi qua giá trị trả về	`if (fd == -1) perror(...)`
Stack frame, call stack	`fork`; mỗi thread một stack
Con trỏ chung `void *`	Đối số của `pthread_create`

Thông điệp chính

Lập trình hệ thống là làm việc với *địa chỉ*: mọi system call cần "trả về" dữ liệu đều nhận một con trỏ từ người gọi.

Bố cục bộ nhớ của một tiến trình



Vùng	Chứa gì – sống bao lâu
Stack	Tham số, biến cục bộ, địa chỉ trả về – *mất khi hàm trả về*
Heap	Vùng `malloc` – sống đến khi `free`
BSS	Biến toàn cục/`static` chưa khởi tạo (tự bằng 0)
Data	Biến toàn cục/`static` có khởi tạo
Text	Mã máy; `rodata`: hằng chuỗi `"abc"` (chỉ đọc)

Đã gặp ở Lab 2

"Biến cục bộ nằm trên stack, mang giá trị rác" và "stack mặc định 8 MB (`ulimit -s`)". Hôm nay ta học cả hai vùng còn lại: **heap** và cách stack *lớn lên* khi gọi hàm.

Thí nghiệm: biến nằm ở đâu? (addr.c)

```c
int g_init = 42;      /* data  */
int g_zero;           /* bss   */

int main(void) {
    static int s = 1;             /* data  */
    int local = 7;                /* stack */
    int *heap = malloc(sizeof *heap);  /* heap */
    if (heap == NULL) return 1;
    printf("rodata: %p\n", (void *)"abc");
    printf("data  : %p\n", (void *)&g_init);
    printf("bss   : %p\n", (void *)&g_zero);
    printf("heap  : %p\n", (void *)heap);
    printf("stack : %p\n", (void *)&local);
    free(heap);
    return 0;
}
```

```
$ gcc -Wall -g addr.c -o addr && ./addr
rodata: 0x5603c8a1a004
data  : 0x5603c8a1c010
bss   : 0x5603c8a1c01c
heap  : 0x5603c9e3b2a0
stack : 0x7ffd5e1c3a34
```

Kết quả minh họa trên Ubuntu x86-64; địa chỉ đổi mỗi lần chạy (ASLR).

- Thứ tự luôn là rodata $<$ data $<$ bss $<$ heap $\ll$ stack.
- %p cần đối số kiểu `void *` – vì thế có ép kiểu.
- Chạy 2 lần: địa chỉ khác nhau, *khoảng cách* giữa data và bss giữ nguyên.

Nội dung

1. Mở đầu: từ Lab 2 sang Lab 3
2. Hàm
3. Call stack và đệ quy
4. Con trỏ
5. Con trỏ, mảng, chuỗi và tìm kiếm nhị phân
6. Cấp phát bộ nhớ động
7. Lỗi bộ nhớ và Valgrind
8. Danh sách liên kết đơn
9. Bài tập thực hành
10. Tổng kết

Vì sao cần hàm?

Không có hàm (`nofun.c`)

```c
double pi = 3.14159;
double rA = 5.0;
printf("A = %.2f\n", pi * rA * rA);
double rB = 3.0;
printf("B = %.2f\n", pi * rB * rB);
double rC = 7.1;
printf("C = %.2f\n", pi * rC + rC);  /* LOI */
```

Có hàm (`circle.c`)

```c
double area(double r) {
    return 3.14159 * r * r;
}

int main(void) {
    printf("A = %.2f\n", area(5.0));
    printf("B = %.2f\n", area(3.0));
    printf("C = %.2f\n", area(7.1));
    return 0;
}
```

Bốn lợi ích

Viết một lần – công thức chỉ ở một chỗ, sửa một chỗ. **Ít lỗi** – lỗi `pi * rC + rC` không thể xảy ra khi không còn chép tay. **Dễ đọc** – `main` đọc như một câu chuyện. **Kiểm thử được** – thử riêng `area()` trước khi dùng khắp nơi.

Khai báo – định nghĩa – lời gọi

```c
#include <stdio.h>
double area(double r);        /* 1. KHAI BAO   */

int main(void) {
    printf("%.2f\n", area(5.0)); /* 3. GOI */
    return 0;
}
double area(double r) {       /* 2. DINH NGHIA */
    return 3.14159 * r * r;
}
```

- **Khai báo (prototype)**: kiểu trả về, tên, kiểu tham số – cho trình biên dịch *kiểm tra* lời gọi.
- **Định nghĩa**: thân hàm, *đúng một lần*. Không trả gì: `void`.

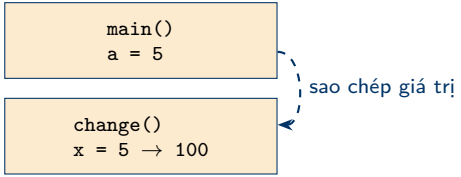
```
$ gcc -Wall -std=c11 noproto.c     # da XOA dong prototype
noproto.c:4:22: warning: implicit declaration of function 'area'
noproto.c:8:8: error: conflicting types for 'area'; have 'double(double)'
```

Đọc lỗi: thiếu prototype, gcc đoán `area` trả về `int`, tới định nghĩa lại thấy `double` ⇒ "conflicting types" (từ gcc 14, dòng đầu đã là lỗi). `#include <stdio.h>` của Lab 2 chính là chép prototype của `printf` vào file.

Truyền tham trị: tham số là *bản sao*

```c
void change(int x) {
    x = 100;           /* sua BAN SAO */
}

int main(void) {
    int a = 5;
    change(a);
    printf("%d\n", a);    /* van la 5 */
    return 0;
}
```



- x là biến *riêng* trong frame của change, khởi tạo bằng giá trị của a.
- Hàm trả về ⇒ frame bị huỷ, x biến mất, a không đổi.

Quy tắc duy nhất của C

Mọi đối số đều truyền tham trị. Muốn hàm sửa biến của người gọi ⇒ truyền **giá trị của địa chỉ** (mục Con trỏ).

Phạm vi và thời gian sống của biến

Loại	Khai báo ở	Ai thấy (phạm vi)	Sống bao lâu	Vùng nhớ
Toàn cục	Ngoài mọi hàm	Cả file	Suốt chương trình	data / bss
Cục bộ, tham số	Trong hàm	Trong hàm	Một lần gọi	stack
Khối	Trong { }	Đến }	Đến }	stack
`static` cục bộ	Trong hàm + `static`	Trong hàm	Suốt chương trình	data / bss

```c
void counter(void) {
    static int count = 0;  /* chi 1 LAN */
    count++;
    printf("Count = %d\n", count);
}
/* goi 3 lan -> in 1, 2, 3 */
```

Phạm vi ≠ thời gian sống

`count` chỉ *thấy được* trong `counter` nhưng *sống* suốt chương trình. Bỏ chữ `static` ⇒ in `1, 1, 1`.

Thói quen: khai báo biến ở phạm vi nhỏ nhất – `for (int i = 0; ...)`; hạn chế biến toàn cục.

Thiết kế hàm: ba quy tắc cho Bài 1–5

1. **Mỗi hàm một việc**, tên là động từ: `electricity_cost`, `print_bill`, `is_leap`.
2. **Tách tính toán khỏi nhập/xuất**: hàm tính toán không `printf`/`scanf` ⇒ gọi lại được, kiểm thử được.
3. **Báo lỗi bằng giá trị trả về** – đúng kiểu system call: `-1`, `false`, `NULL`.

```c
long long electricity_cost(int kwh);
void print_bill(int kwh, long long cost);

int main(void) {
    int kwh;
    if (scanf("%d", &kwh) != 1 || kwh < 0) {
        fprintf(stderr, "Loi: ...\n");
        return 1;
    }
    /* main chi DIEU PHOI */
    print_bill(kwh, electricity_cost(kwh));
    return 0;
}
```

Thay đổi so với Lab 2

Ở Lab 2 mọi lời giải nằm gọn trong `main`. Từ tuần này, "*không viết toàn bộ xử lý trong* `main()`" (đề Bài 1) là một tiêu chí chấm: `main` chỉ đọc dữ liệu, gọi hàm và in kết quả.

Nội dung

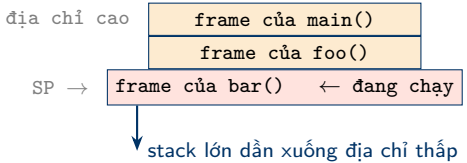
1. Mở đầu: từ Lab 2 sang Lab 3
2. Hàm
3. Call stack và đệ quy
4. Con trỏ
5. Con trỏ, mảng, chuỗi và tìm kiếm nhị phân
6. Cấp phát bộ nhớ động
7. Lỗi bộ nhớ và Valgrind
8. Danh sách liên kết đơn
9. Bài tập thực hành
10. Tổng kết

Stack frame: không gian làm việc của *một lần* gọi hàm

```c
void bar(void) { /* sau nhat */ }
void foo(void) { bar(); }
int main(void) {
    foo();      /* main -> foo -> bar */
    return 0;
}
```

Mỗi frame chứa:

- tham số (bản sao) và biến cục bộ,
- **địa chỉ trả về** – quay lại đâu trong hàm gọi,
- thanh ghi đã lưu của hàm gọi.



- Gọi hàm: **push** một frame. Hàm trả về: **pop** frame đó (LIFO).
- SP (stack pointer) chỉ đỉnh stack; FP (frame pointer) mốc của frame hiện tại.
- Vì frame bị pop nên *biến cục bộ mất khi hàm trả về*.

Quan sát call stack bằng gdb – lệnh bt của Lab 2

```c
#include <stdio.h>

int factorial(int n) {
    if (n <= 1)
        return 1;
    return n * factorial(n - 1);
}

int main(void) {
    printf("%d\n", factorial(4));
    return 0;
}
```

```
$ gcc -g -Wall fact.c -o fact && gdb -q ./fact
(gdb) break factorial if n == 1
(gdb) run
Breakpoint 1, factorial (n=1) at fact.c:4
(gdb) bt
#0  factorial (n=1) at fact.c:4
#1  0x...  in factorial (n=2) at fact.c:6
#2  0x...  in factorial (n=3) at fact.c:6
#3  0x...  in factorial (n=4) at fact.c:6
#4  0x...  in main () at fact.c:10
(gdb) frame 2
(gdb) print n
$1 = 3
```

Đọc kết quả: 4 lời gọi `factorial` đang *cùng sống*, mỗi lời gọi một biến `n` riêng (1, 2, 3, 4). `frame k` chuyển sang frame thứ *k* để xem biến của lời gọi đó. Điểm dừng có điều kiện `if n == 1` dừng đúng ở *đáy* đệ quy.

Hệ quả: không trả về địa chỉ biến cục bộ

```
int *make_array(void) {
    int local[100];
    local[0] = 1;
    return local;        /* SAI: frame cua make_array se bi huy */
}
```

```
$ gcc -Wall -c traps.c
traps.c:6:12: warning: function returns address of local variable [-Wreturn-local-addr]
```

Con trỏ "treo" (dangling pointer)

Khi `make_array` trả về, vùng `local` thuộc về lời gọi hàm *tiếp theo* và bị ghi đè bất cứ lúc nào. gcc 13 còn biên dịch hàm này thành trả về `NULL` $\Rightarrow$ segfault ở chỗ dùng con trỏ.

Hai cách đúng: (1) người gọi cấp vùng nhớ rồi truyền vào, `void fill(int *a, int n)` – đúng kiểu `read(fd, buf, n)`; (2) cấp phát trên heap bằng `malloc`, người gọi `free` (mục Cấp phát động).

Đệ quy: điểm dừng + bước tiến

```c
int factorial(int n) {
    if (n <= 1)          /* DIEM DUNG */
        return 1;
    return n * factorial(n - 1);
}                        /* BUOC TIEN */
```

Hai điều kiện bắt buộc

1. Điểm dừng (base case): trả kết quả không cần gọi lại.
2. Bước tiến: mỗi lời gọi *tiến gần* điểm dừng hơn.

```
factorial(4)
= 4 * factorial(3)              <- di xuong:
= 4 * (3 * factorial(2))           moi buoc
= 4 * (3 * (2 * factorial(1)))     push 1 frame
= 4 * (3 * (2 * 1))             <- cham day
= 4 * (3 * 2)                   <- di len:
= 4 * 6                            moi buoc
= 24                               pop 1 frame
```

Cách thiết kế: giả sử lời gọi với bài toán nhỏ hơn *đã đúng*, chỉ lo hai việc: kết hợp kết quả đó, và xử lý trường hợp nhỏ nhất. Đây là cách giải Bài 6–10.

Stack overflow: khi đệ quy không dừng hoặc quá sâu

```
int factorial(int n) {
    return n * factorial(n - 1);
}   /* THIEU diem dung:
       f(4) f(3) ... f(0) f(-1) ... */
```

```
$ ./broken
Segmentation fault (core dumped)
$ ulimit -s
8192          <- stack 8 MB
```

- Mỗi lời gọi `factorial` ở -O0 tốn cỡ 32 byte stack $\Rightarrow$ khoảng 8 MB / 32 B $\approx$ 260 000 tầng là hết (ước lượng).
- Hàm có mảng cục bộ lớn thì hết nhanh hơn rất nhiều.
- Trong gdb: `run` $\rightarrow$ dừng ở SIGSEGV $\rightarrow$ `bt 5` thấy hàng trăm nghìn frame giống nhau.

Ba nguyên nhân

Thiếu điểm dừng – bước đệ quy không tiến gần điểm dừng – độ sâu quá lớn (Bài 6 với $n = 10^6$).

Đệ quy hay vòng lặp? Fibonacci – khép lại bài về nhà của Lab 1

```c
long long fib_rec(int n) {      /* O(2^n) */
    if (n < 2) return n;
    return fib_rec(n - 1) + fib_rec(n - 2);
}

long long fib_iter(int n) {     /* O(n)   */
    long long a = 0, b = 1;
    for (int i = 0; i < n; i++) {
        long long t = a + b;
        a = b;
        b = t;
    }
    return a;
}
```

n	số lời gọi `fib_rec`	thời gian
30	2 692 537	$\approx$ 0,003 s
35	29 860 703	$\approx$ 0,04 s
40	331 160 281	$\approx$ 0,45 s
45	3 672 623 805	$\approx$ 5 s

Đo bằng `clock()`, `-O0`, trên máy soạn bài; `fib_iter` cho mọi n ở trên: $<$ 1 micro giây.

Vì sao?

`fib_rec(n-1)` và `fib_rec(n-2)` *tính lại* cùng các giá trị hàng triệu lần: số lời gọi $= 2F(n+1) - 1$, tăng theo $\approx 1{,}618^n$.

Đính chính Lab 1: số nguyên 64 bit có dấu (`long long`, cũng là số của bash) giữ đúng tới $F(92)$; $F(93)$ *đã* tràn (bash in ra số âm). Slide Lab 1 ghi nhầm là $F(93)$ còn đúng.

Chọn kiểu dữ liệu cho kết quả đệ quy

Kiểu	Giá trị lớn nhất	Giai thừa đúng tới	Fibonacci đúng tới
`int` (32 bit)	2 147 483 647	12!	$F(46)$
`long long` (64 bit)	$\approx 9{,}22 \times 10^{18}$	20!	$F(92)$
`unsigned long long`	$\approx 1{,}84 \times 10^{19}$	20!	$F(93)$

Tràn số không báo lỗi

Với `int`, `factorial(13)` trả về 1932053504 (đúng phải là 6227020800) – không cảnh báo, không crash. Tràn số có dấu là *hành vi không xác định*; `-fsanitize=undefined` (Lab 2) bắt được tại chỗ.

Bài làm bền vững

Chọn kiểu đủ rộng **và** giới hạn miền nhập: Bài 7 chỉ nhận $0 \leq n \leq 20$ và nói rõ lý do trong thông báo lỗi.

Nội dung

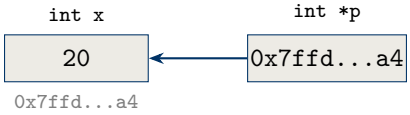
1. Mở đầu: từ Lab 2 sang Lab 3
2. Hàm
3. Call stack và đệ quy
4. Con trỏ
5. Con trỏ, mảng, chuỗi và tìm kiếm nhị phân
6. Cấp phát bộ nhớ động
7. Lỗi bộ nhớ và Valgrind
8. Danh sách liên kết đơn
9. Bài tập thực hành
10. Tổng kết

Con trỏ là một biến chứa *địa chỉ*

```c
int x = 10;
int *p = &x;          /* p giu dia chi cua x */

printf("%d\n", *p);   /* 10: DOC qua p       */
*p = 20;              /* GHI qua p           */
printf("%d\n", x);    /* 20                  */

/* in cung mot dia chi hai lan: */
printf("%p %p\n", (void *)p, (void *)&x);
```



`&x`	địa chỉ của `x`
`int *p`	(khai báo) `p` là con trỏ tới `int`
`*p`	(biểu thức) ô nhớ mà `p` trỏ tới

Cần thuộc: khi `p = &x` thì `*p` *chính là* `x`, và `*(&x) == x`. Kiểu `int *` cho biết `*p` đọc *bao nhiêu byte* và *hiểu* chúng thế nào – đúng vai trò của kiểu ở Lab 2.

Khởi tạo con trỏ: ba cách đúng, ba lỗi kinh điển

Đúng

```c
int x = 10;
int *p1 = &x;                       /* 1 */
int *p2 = NULL;                     /* 2 */
int *p3 = malloc(sizeof *p3);       /* 3 */
if (p3 == NULL) { /* bao loi */ }
```

Sai

```c
int *w;       /* wild: chua khoi tao */
*w = 10;      /* ghi vao dia chi RAC */

int *n = NULL;
*n = 5;       /* Segmentation fault  */

free(p3);
*p3 = 7;      /* dangling: da tra lai */
```

- `NULL` nghĩa là "chưa trỏ tới đâu" – *kiểm tra được*: `if (p != NULL)`. Con trỏ rác thì không kiểm tra được.
- `gcc -Wall` cảnh báo `'w' is used uninitialized`; với p3 sau `free`, gcc $\geq$ 12 báo `pointer 'p3' used after 'free'`.
- Ba lỗi này đều là *hành vi không xác định*: có thể chạy "đúng" trên máy bạn và sai trên máy chấm.

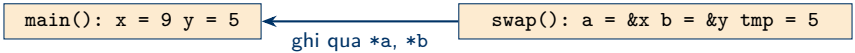
Bẫy truyền tham trị: swap

Sai – đổi hai *bản sao*

```
void swap(int a, int b) {
    int tmp = a;
    a = b;
    b = tmp;
}
int x = 5, y = 9;
swap(x, y);          /* x = 5, y = 9 */
```

Đúng – đổi qua *địa chỉ* (Bài 11)

```
void swap(int *a, int *b) {
    int tmp = *a;    /* doc o nho a tro toi */
    *a = *b;         /* ghi vao o nho do    */
    *b = tmp;
}
int x = 5, y = 9;
swap(&x, &y);        /* x = 9, y = 5 */
```



Vẫn là truyền tham trị: a, b là *bản sao địa chỉ* – nhưng bản sao địa chỉ vẫn trỏ đúng vào x, y của main.

Tham số đầu ra: một lời gọi, nhiều kết quả

```c
/* a, b: dau vao.  sum, diff: noi GHI ket qua */
void sum_diff(int a, int b, int *sum, int *diff) {
    *sum  = a + b;
    *diff = a - b;
}

int s, d;
sum_diff(10, 4, &s, &d);  /* s = 14, d = 6 */
```

```c
/* Bai 14: tra ve TRANG THAI,
   sua so du qua con tro            */
bool withdraw(long long *balance, long long x) {
    if (x <= 0 || x > *balance) return false;
    *balance -= x;
    return true;
}
```

Mẫu thiết kế của system call

Giá trị trả về báo thành công/thất bại; **kết quả thật** ghi qua con trỏ do người gọi cấp.

- `scanf("%d", &x)`: trả về số trường đọc được, giá trị ghi vào `x`.
- `wait(&status)` (tuần 7): trả về PID, trạng thái ghi vào `status`.
- `min_max(a, n, &lo, &hi)` – Bài 15.

Đọc một khai báo con trỏ

Khai báo	Đọc là	Gặp ở đâu
`int *p`	p là con trỏ tới `int`	`swap`, `scanf`
`const int *p`	con trỏ tới `int` *không được sửa qua* p	Hàm chỉ đọc mảng (`max_of`)
`int **pp`	con trỏ tới (con trỏ tới `int`)	Sửa con trỏ của người gọi (`insert_head`)
`void *p`	con trỏ "chung", chưa biết kiểu	Giá trị trả về của `malloc`
`char *argv[]`	mảng các con trỏ tới `char`	Tham số dòng lệnh (`$1`, `$2` của Lab 1)

Mẹo

Đọc từ tên biến ra ngoài: `const int *p` → "p là con trỏ tới `int` hằng". Dấu `*` *trong khai báo* nghĩa là "là con trỏ"; *trong biểu thức* nghĩa là "ô được trỏ tới".

`int* a, b;` khai báo một con trỏ và một `int`

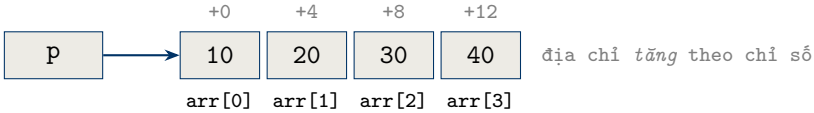
`*` gắn với tên biến, không gắn với kiểu. Viết `int *a, *b;` hoặc mỗi dòng một biến.

Nội dung

1. Mở đầu: từ Lab 2 sang Lab 3
2. Hàm
3. Call stack và đệ quy
4. Con trỏ
5. Con trỏ, mảng, chuỗi và tìm kiếm nhị phân
6. Cấp phát bộ nhớ động
7. Lỗi bộ nhớ và Valgrind
8. Danh sách liên kết đơn
9. Bài tập thực hành
10. Tổng kết

Tên mảng "suy biến" thành con trỏ tới phần tử đầu

```
int arr[4] = {10, 20, 30, 40};
int *p = arr;            /* arr suy bien thanh &arr[0] */
printf("%d %d %d\n", *arr, p[1], *(p + 2));   /* 10 20 30 */
```



- `arr[i]` ≡ `*(arr + i)`; `p[i]` ≡ `*(p + i)`.
- `arr` là địa chỉ *cố định*: `arr++` lỗi biên dịch; `p++` hợp lệ.
- `sizeof arr` = 16 (cả mảng); `sizeof p` = 8 (một con trỏ).
- Hình SP03 (tr. 14) ghi `arr[3] = 30` và địa chỉ giảm dần – cần sửa theo hình này.

Số học con trỏ: đơn vị là *phần tử*, không phải byte

```c
int arr[4] = {10, 20, 30, 40};
int *p = arr;
p += 2;                    /* p -> arr[2], +8 byte */
printf("%d\n", *p);        /* 30 */

int *q = &arr[3];
printf("%td\n", q - p);    /* 1: so PHAN TU giua */

int *end = arr + 4;        /* "one past the end" */
for (int *it = arr; it < end; it++)
    printf("%d ", *it);    /* 10 20 30 40 */
```

- `p + i` tiến `i * sizeof *p` byte: 4 với `int`, 8 với `double`.
- `q - p` có kiểu `ptrdiff_t`, in bằng `%td`.
- So sánh `<`, `>` chỉ có nghĩa khi hai con trỏ cùng trỏ vào *một* mảng.
- `arr + 4` được phép *tính* và *so sánh* nhưng **không được** `*`.
- Lab 2, Bài 14: `strstr(s, sub) - s` chính là phép trừ con trỏ → vị trí chuỗi con.

Truyền mảng vào hàm và bẫy `sizeof`

```c
/* Ba khai bao nay GIONG HET nhau: */
void f(int *arr, int n);
void f(int arr[], int n);
void f(int arr[10], int n);  /* 10 bi bo qua */

void process(int arr[]) {
    int n = sizeof(arr) / sizeof(arr[0]); /* SAI */
}
```

```
warning: 'sizeof' on array function parameter 'arr'
        will return size of 'int *' [-Wsizeof-array-argument]
```

- Truyền mảng = truyền *địa chỉ phần tử đầu*; số phần tử **bị mất**.
- Trong hàm, `sizeof(arr)` $= 8$ (con trỏ) $\Rightarrow n = 8/4 = 2$: sai mà không crash.
- Hàm *sửa được* mảng gốc (không có bản sao mảng).

Thành ngữ chuẩn của C

Luôn truyền thêm `n`; thêm `const` khi hàm chỉ đọc: `int max_of(const int *a, int n);`

Con trỏ và chuỗi

```c
char name[] = "Alex";
/* duyet chuoi bang con tro */
for (char *p = name; *p != '\0'; p++)
    putchar(*p);

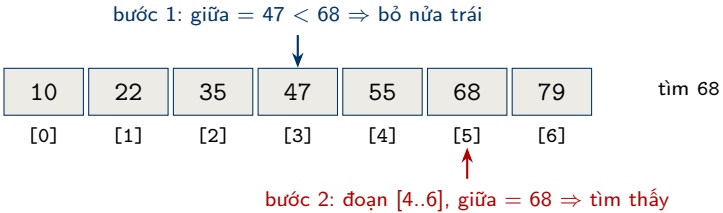
char  arr[] = "Hello"; /* mang: sua duoc  */
char *s     = "Hello"; /* rodata: CHI DOC */
arr[0] = 'h';          /* OK              */
s[0]   = 'h';          /* Segfault        */

/* dung y: t[0] = 'h' -> LOI bien dich */
const char *t = "Hello";
```

- `char arr[]` *sao chép* chuỗi vào một mảng của bạn.
- `char *s` chỉ *trỏ* vào hằng chuỗi trong vùng rodata (thí nghiệm `addr.c`).
- Thói quen: con trỏ tới hằng chuỗi luôn khai báo `const char *` – như `name_of()` của Bài 3.
- Đi quá `'\0'` là đọc ngoài vùng nhớ (*buffer over-read*) – gốc của lỗ hổng Heartbleed.

Tìm kiếm nhị phân: ý tưởng

Điều kiện: mảng *đã sắp xếp* (sắp xếp học ở tuần sau). Mỗi bước so với phần tử giữa và bỏ đi một nửa.



n	tuyến tính	nhị phân
1 000	1 000 bước	10 bước
1 000 000	10^6 bước	20 bước
10^9	10^9 bước	30 bước

Nối với Lab 2, Bài 11

Tìm tuyến tính: $O(n)$, chạy trên mọi mảng. Tìm nhị phân: $O(\log_2 n)$, nhưng *bắt buộc* mảng đã sắp xếp. Chi phí sắp xếp trả một lần; mỗi lần tìm sau đó đều được lợi.

Tìm kiếm nhị phân: hai cách cài đặt

Chỉ số, vòng lặp – đoạn đóng `[lo, hi]`

```c
int bsearch_idx(const int *a, int n, int key) {
    int lo = 0, hi = n - 1;
    while (lo <= hi) {
        int mid = lo + (hi - lo) / 2;
        if (a[mid] == key) return mid;
        if (a[mid] < key)  lo = mid + 1;
        else               hi = mid - 1;
    }
    return -1;                /* khong thay */
}
```

Con trỏ, đệ quy – đoạn nửa mở `[lo, hi)`

```c
const int *bsearch_ptr(const int *lo,
                       const int *hi, int key) {
    if (lo >= hi) return NULL;      /* doan rong */
    const int *mid = lo + (hi - lo) / 2;
    if (*mid == key) return mid;
    if (*mid < key)
        return bsearch_ptr(mid + 1, hi, key);
    return bsearch_ptr(lo, mid, key); /* KHONG -1 */
}
/* goi: bsearch_ptr(a, a + n, key) */
```

Hai chi tiết mà tài liệu Lab và SP03 cần sửa

`(lo + hi) / 2` tràn số khi mảng rất lớn → dùng `lo + (hi - lo) / 2`. Bản con trỏ dùng `mid - 1` sẽ tạo con trỏ *đứng trước* `a[0]` khi `mid == lo` – hành vi không xác định; đoạn nửa mở tránh được hoàn toàn.

Nội dung

1 Mở đầu: từ Lab 2 sang Lab 3

2 Hàm

3 Call stack và đệ quy

4 Con trỏ

5 Con trỏ, mảng, chuỗi và tìm kiếm nhị phân

6 Cấp phát bộ nhớ động

7 Lỗi bộ nhớ và Valgrind

8 Danh sách liên kết đơn

9 Bài tập thực hành

10 Tổng kết

Ba giới hạn của stack ⇒ cần heap

```c
/* 1. Kich thuoc chi biet luc chay */
int n;  scanf("%d", &n);
int a[n];       /* VLA (Lab 2): khong bao
                   duoc khi het cho */

/* 2. Du lieu chet theo ham */
int *make(void) {
    int local[100];
    return local;       /* dangling */
}

/* 3. Stack nho (8 MB) */
int big[10000000];      /* 40 MB -> crash */
```

	Stack	**Heap**
Kích thước	∼8 MB	đến hết RAM
Cấp/thu hồi	tự động	`malloc`/`free`
Sống đến	hết hàm	khi `free`
Báo thất bại	không (crash)	trả `NULL`

Khép lời hứa của Lab 2

"Tuần sau sẽ học `malloc` để giải quyết triệt để" bài toán *kích thước chỉ biết lúc chạy* (Bài 10 Lab 2 dùng VLA `names[n][20]`).

Bốn hàm của <stdlib.h>

Hàm	Làm gì	Nội dung ban đầu	Khi thất bại
`malloc(size)`	cấp `size` byte	**rác**	trả `NULL`
`calloc(n, size)`	cấp `n * size` byte	**toàn 0**	trả `NULL`
`realloc(p, size)`	đổi kích thước, *có thể dời chỗ*	phần cũ giữ nguyên, phần thêm là rác	trả `NULL`, **vùng cũ vẫn còn**
`free(p)`	trả vùng nhớ cho bộ cấp phát	–	`free(NULL)` không làm gì

- Kiểu trả về là `void *`: gán thẳng cho `int *`, `double *`... – trong C *không cần* ép kiểu `(int *)`.
- `calloc` còn kiểm tra tràn số của phép nhân `n * size`.

Quy tắc vàng

Mỗi vùng cấp phát *thành công* phải được `free` **đúng một lần**. Ít hơn: rò rỉ. Nhiều hơn: *double free*.

Mẫu chuẩn năm bước (Bài 16)

```c
/* 1 */ double *scores =
            malloc((size_t)n * sizeof *scores);
/* 2 */ if (scores == NULL) {
            perror("malloc");
            return 1;
        }
/* 3 */ for (int i = 0; i < n; i++) {
            if (scanf("%lf", &scores[i]) != 1) {
                free(scores);   /* nhanh loi: free */
                return 1;
            }
        }
        /* ... tinh trung binh ... */
/* 4 */ free(scores);
/* 5 */ scores = NULL;
```

1. **Cấp phát**: `sizeof *scores` thay vì `sizeof(double)` – đổi kiểu không phải sửa dòng này.
2. **Kiểm tra** `NULL`; `perror` in lý do từ `errno`.
3. **Dùng** như mảng thường: `scores[i]`.
4. **Giải phóng** trên *mọi* đường ra, kể cả nhánh lỗi.
5. **Gán** `NULL` để lỡ dùng lại thì crash ngay, dễ tìm.

realloc an toàn (Bài 18)

Sai

```
t = realloc(t, (n + m) * sizeof *t);
/* that bai: t = NULL, mat dia chi vung
   cu -> RO RI va mat ca du lieu cu    */
```

Đúng

```
double *tmp =
    realloc(t, (size_t)(n + m) * sizeof *t);
if (tmp == NULL) {
    perror("realloc");
    free(t);        /* vung cu VAN CON  */
    return 1;
}
t = tmp;            /* gan khi thanh cong */
```

- `realloc` có thể *dời* khối sang chỗ mới (chép dữ liệu cũ, giải phóng chỗ cũ). Mọi con trỏ *khác* còn giữ địa chỉ cũ đều thành dangling.
- Phần mở rộng `t[n]` ... `t[n+m-1]` chứa rác – phải ghi trước khi đọc.
- `realloc(NULL, size)` ≡ `malloc(size)`.

SP03 tr. 22 ghi "If fails, the original ptr is lost"

Không chính xác: khi thất bại vùng cũ còn nguyên; nó chỉ "mất" nếu ta gán đè như bản sai bên trái.

malloc không phải system call

```
$ strace -e trace=brk,mmap ./bigalloc
...
brk(NULL)                       = 0x5581e2b5e000
brk(0x5581e2b7f000)             = 0x5581e2b7f000
mmap(NULL, 8392704, PROT_READ|PROT_WRITE,
     MAP_PRIVATE|MAP_ANONYMOUS, -1, 0)
                                = 0x7f3c1a1ff000
```

Minh họa rút gọn: `malloc(100)` rồi `malloc(8000000)`; số cụ thể khác trên máy bạn.

- `malloc` là hàm **thư viện** (glibc, `man 3 malloc`): tự chia nhỏ vùng heap đã có.
- Chỉ khi hết chỗ mới gọi kernel: `brk` nới rộng heap (khối nhỏ), `mmap` cho khối lớn (từ 128 KiB).
- `free` trả vùng nhớ cho *bộ cấp phát*, thường chưa trả ngay cho hệ điều hành.

Giống `printf` và `write` ở Lab 2

Hàm mục 3 bọc quanh system call mục 2. `strace` cho thấy ranh giới.

Ai cấp phát, ai giải phóng? (Bài 19)

```c
/* Tao mang moi = a noi tiep b.
   Nguoi GOI phai free() ket qua. */
int *merge(const int *a, int na,
           const int *b, int nb) {
    int *c = malloc((size_t)(na + nb) * sizeof *c);
    if (c == NULL) return NULL;
    memcpy(c, a, (size_t)na * sizeof *a);
    memcpy(c + na, b, (size_t)nb * sizeof *b);
    return c;          /* vung heap song tiep */
}
```

- Vùng heap *sống qua* lệnh `return` – điều stack không làm được.
- Hàm trả về vùng `malloc` phải **ghi rõ trong chú thích** ai `free`.
- Đếm: số lần `malloc` thành công = số lần `free`. Bài 19: 3 và 3.

Đếm để kiểm chứng

Valgrind in `total heap usage: A allocs, F frees`: chương trình sạch khi $A = F$ (con số gồm cả bộ đệm mà `printf`/`scanf` tự cấp phát).

Nội dung

1. Mở đầu: từ Lab 2 sang Lab 3
2. Hàm
3. Call stack và đệ quy
4. Con trỏ
5. Con trỏ, mảng, chuỗi và tìm kiếm nhị phân
6. Cấp phát bộ nhớ động
7. Lỗi bộ nhớ và Valgrind
8. Danh sách liên kết đơn
9. Bài tập thực hành
10. Tổng kết

Sáu lỗi bộ nhớ kinh điển

Lỗi	Ví dụ	Biểu hiện	Ai bắt được
Rò rỉ (leak)	quên `free`	RAM tăng dần; chương trình ngắn thì "không sao"	Valgrind, ASan
Dùng sau khi free	`free(p); *p = 1;`	sai ngẫu nhiên, đôi khi crash	gcc $\geq$ 12 (cùng hàm), Valgrind, ASan
Giải phóng hai lần	`free(p); free(p);`	glibc dừng: `double free detected`	Valgrind, ASan
Dereference `NULL`	không kiểm tra `malloc`	`Segmentation fault`	gdb (dừng đúng dòng)
Đọc vùng chưa khởi tạo	`malloc` rồi đọc ngay	kết quả thay đổi giữa các lần chạy	Valgrind
Ghi ngoài vùng cấp phát	`a[n] = 0;` với `n` phần tử	hỏng heap, crash ở chỗ khác	Valgrind, ASan

Vì sao cần công cụ

Cả sáu lỗi đều có thể "chạy đúng" khi bạn thử. Không thể chứng minh chương trình sạch lỗi bộ nhớ bằng cách nhìn output – phải đo bằng công cụ.

Bốn nguyên nhân rò rỉ thường gặp

```c
/* 1. Quen free */
int *p = malloc(100 * sizeof *p);
/* ... dung p ... het ham */

/* 2. Ghi de con tro truoc khi free */
int *q = malloc(40);
q = malloc(80);      /* 40 byte cu mat dau */
```

```c
/* 3. Thoat som o nhanh loi */
int *r = malloc(n * sizeof *r);
if (scanf("%d", &r[0]) != 1)
    return 1;        /* quen free(r) */

/* 4. Cap phat trong vong lap */
for (int i = 0; i < 1000; i++) {
    int *t = malloc(400);
    /* ... khong free(t) ... */
}
```

Nguyên nhân số 3 hay gặp nhất trong Bài 16–20

Mỗi `return` giữa chừng là một "đường ra". Trước mỗi đường ra, tự hỏi: *vùng nào đã cấp phát mà chưa giải phóng?*

Valgrind: chạy và đọc báo cáo rò rỉ

```c
#include <stdlib.h>

void bad(void) {
    int *p = malloc(10 * sizeof *p);
    if (p == NULL) return;
    p[0] = 100;
}           /* quen free(p) */

int main(void) {
    bad();
    return 0;
}
```

```
sudo apt install valgrind    # 1 lan
gcc -g -Wall leak.c -o leak  # -g: co so dong
valgrind --leak-check=full ./leak
```

```
==4321== HEAP SUMMARY:
==4321==     in use at exit: 40 bytes in 1 blocks
==4321==   total heap usage: 1 allocs, 0 frees, 40 bytes allocated
==4321==
==4321== 40 bytes in 1 blocks are definitely lost in loss record 1 of 1
==4321==    at 0x4846828: malloc (vg_replace_malloc.c:...)
==4321==    by 0x109165: bad (leak.c:4)
==4321==    by 0x10918A: main (leak.c:10)
==4321==
==4321== LEAK SUMMARY:
==4321==    definitely lost: 40 bytes in 1 blocks
==4321==    indirectly lost: 0 bytes in 0 blocks
==4321==      possibly lost: 0 bytes in 0 blocks
==4321==    still reachable: 0 bytes in 0 blocks
==4321== ERROR SUMMARY: 1 errors from 1 contexts
```

Rút gọn; PID và địa chỉ khác trên máy bạn. **Đọc từ trên xuống:** `malloc` được gọi ở `leak.c:4` trong `bad`; `bad` được `main` gọi ở dòng 10. Chương trình sạch khi thấy: `All heap blocks were freed -- no leaks are possible`.

Valgrind: lỗi truy cập bộ nhớ

```
1 #include <stdlib.h>
2 int main(void) {
3     int *p = malloc(sizeof *p)
      ;
4     if (p == NULL) return 1;
5     free(p);
6     *p = 10;       /* UAF */
7     return 0;
8 }
9
```

Có `free(p)` thêm một lần ở dòng 7 ⇒ Valgrind báo `Invalid free()`; không có Valgrind, glibc dừng chương trình với `free(): double free detected`.

```
==4321== Invalid write of size 4
==4321==    at 0x109193: main (uaf.c:6)
==4321==  Address 0x4a8a040 is 0 bytes inside a block of size 4 free'd
==4321==    at 0x484988F: free (vg_replace_malloc.c:...)
==4321==    by 0x10918E: main (uaf.c:5)
==4321==  Block was alloc'd at
==4321==    at 0x4846828: malloc (vg_replace_malloc.c:...)
==4321==    by 0x10917E: main (uaf.c:3)
```

Thông báo của Valgrind	**Nghĩa là**
`Invalid write/read of size N`	ghi/đọc N byte ở chỗ không được phép (sau `free`, ngoài biên)
`Invalid free()`	`free` hai lần, hoặc `free` địa chỉ không lấy từ `malloc`
`Conditional jump ... uninitialised`	`if` dựa trên giá trị chưa khởi tạo (thêm `--track-origins=yes`)
`definitely lost`	rò rỉ chắc chắn: không còn con trỏ nào tới khối
`indirectly lost`	mất vì khối *chứa* con trỏ tới nó đã mất (danh sách liên kết)

Valgrind hay AddressSanitizer? Và gcc bắt được gì trước?

	Valgrind	**ASan (Lab 2)**
Cách dùng	`valgrind ./prog`, không cần biên dịch lại	biên dịch lại với `-fsanitize=address`
Tốc độ	chậm 20–50 lần	chậm ∼2 lần
Bắt leak	có (`--leak-check=full`)	có (LeakSanitizer)
Đọc chưa khởi tạo	**có**	không
Ngoài biên mảng *stack*	không	**có**

Không dùng chung: chương trình biên dịch với ASan không chạy đúng dưới Valgrind. Mỗi lần chạy chọn một trong hai.

```
$ gcc -Wall -c traps.c
warning: function returns address
  of local variable [-Wreturn-local-addr]
warning: 'sizeof' on array function
  parameter 'arr' will return size
  of 'int *' [-Wsizeof-array-argument]
warning: pointer 'p' used after
  'free' [-Wuse-after-free]
```

`-Wall` bắt được lỗi *nằm gọn trong một hàm*. Khi `malloc` ở hàm này, `free` ở hàm khác, chỉ công cụ lúc chạy mới thấy.

Quy trình khi nghi có lỗi bộ nhớ

1. Đọc hết cảnh báo của `gcc -Wall -Wextra -g`.
2. Chạy với input nhỏ nhất gây sai, dưới `valgrind --leak-check=full`.
3. Sửa lỗi **đầu tiên** trong báo cáo trước – các lỗi sau thường là hệ quả.
4. Segfault: chạy trong gdb, `bt` để biết dòng, `p` các con trỏ.
5. Chạy lại đến khi thấy `All heap blocks were freed` và `ERROR SUMMARY: 0 errors`.

```
(gdb) print p           # gia tri con tro
$1 = (int *) 0x0        # NULL!
(gdb) print *a@5        # a la int*: 5 phan tu
$2 = {6, 7, 8, 9, 10}
(gdb) x/5dw a           # 5 word 4 byte, so nguyen
0x5555555592a0: 6   7   8   9
0x5555555592b0: 10
(gdb) x/5fg scores      # mang double (Bai 16)
(gdb) bt                # ai goi ai
```

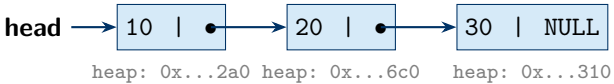
`x/5dw` chỉ đúng với mảng `int`; mảng `double` dùng `x/5fg` (`f`: số thực, `g`: 8 byte), nếu không sẽ thấy số rác. Tra cứu: `man 3 malloc` (mục RETURN VALUE), `man 3 free`, `valgrind --help`.

Nội dung

1. Mở đầu: từ Lab 2 sang Lab 3
2. Hàm
3. Call stack và đệ quy
4. Con trỏ
5. Con trỏ, mảng, chuỗi và tìm kiếm nhị phân
6. Cấp phát bộ nhớ động
7. Lỗi bộ nhớ và Valgrind
8. Danh sách liên kết đơn
9. Bài tập thực hành
10. Tổng kết

Vì sao cần danh sách liên kết?

Chèn vào đầu mảng phải dời mọi phần tử sang phải: $O(n)$, và mảng có kích thước cố định.



- Mỗi **nút** là một vùng `malloc` riêng: dữ liệu + *con trỏ tới nút kế*.
- Các nút nằm *rải rác* trên heap; chỉ cần giữ `head`.
- Nút cuối có `next = NULL`.

Kết hợp cả ba nội dung của buổi hôm nay

`struct` gói dữ liệu, **con trỏ** nối các nút, **cấp phát động** tạo nút khi cần. Phần này *trình diễn*; `struct` và bài thực hành danh sách liên kết thuộc Lab 4.

struct Node và hàm tạo nút

```c
#include <stdlib.h>

struct Node {
    int data;             /* du lieu       */
    struct Node *next;    /* tu tham chieu */
};

struct Node *new_node(int val) {
    struct Node *n = malloc(sizeof *n);
    if (n == NULL)
        return NULL;      /* bao loi       */
    n->data = val;        /* == (*n).data  */
    n->next = NULL;
    return n;
}
```

- `struct` gom nhiều trường thành một kiểu (học kỹ ở Lab 4).
- `struct Node` chứa con trỏ tới *chính kiểu của nó*: cấu trúc **tự tham chiếu** – nền của list, cây, đồ thị.
- `p->x` là cách viết gọn của `(*p).x`.
- `sizeof *n` = 16 trên x86-64: 4 byte `int` + 4 byte đệm + 8 byte con trỏ. Vì sao có 4 byte đệm? → Lab 4.

insert_head: vì sao tham số là struct Node **?

```c
int insert_head(struct Node **head, int val) {
    struct Node *n = new_node(val);
    if (n == NULL)
        return -1;
    n->next = *head;  /* 1. tro vao nut dau cu */
    *head = n;        /* 2. SUA head cua main  */
    return 0;
}

/* trong main: */
struct Node *head = NULL;
insert_head(&head, 30);
insert_head(&head, 20);
insert_head(&head, 10);
/* 10 -> 20 -> 30 -> NULL */
```

Đúng là bài swap, lên một tầng

Muốn hàm sửa `int x` của người gọi ⇒ truyền `int *`.
Muốn hàm sửa `struct Node *head` của người gọi ⇒ truyền `struct Node **`.

- Truyền `head` (không `&`): hàm chỉ sửa *bản sao*, `head` của `main` vẫn là `NULL` và nút mới bị rò rỉ.
- Chèn đầu là $O(1)$: số bước không phụ thuộc độ dài danh sách.

Duyệt, tìm kiếm, xoá, giải phóng

```c
void print_list(const struct Node *p) {
    for (; p != NULL; p = p->next)
        printf("%d -> ", p->data);
    printf("NULL\n");
}

struct Node *search(struct Node *p, int key) {
    for (; p != NULL; p = p->next)
        if (p->data == key) return p;
    return NULL;
}

void free_list(struct Node *head) {
    while (head != NULL) {
        struct Node *tmp = head;  /* 1. giu nut nay */
        head = head->next;        /* 2. sang nut ke */
        free(tmp);                /* 3. roi moi free */
    }
}
```

```c
/* Xoa nut DAU TIEN co gia tri key.
   Tra ve 1 neu da xoa, 0 neu khong thay. */
int delete_value(struct Node **head, int key) {
    struct Node *prev = NULL, *cur = *head;
    while (cur != NULL && cur->data != key) {
        prev = cur;
        cur = cur->next;
    }
    if (cur == NULL) return 0;          /* khong co */
    if (prev == NULL) *head = cur->next; /* nut dau */
    else prev->next = cur->next;    /* giua / cuoi */
    free(cur);
    return 1;
}
```

Thứ tự trong `free_list` là bắt buộc: `free(head)` rồi mới đọc `head->next` là *dùng sau khi free*. Tài liệu Lab chưa có thao tác xoá – bổ sung ở đây.

Chi phí các thao tác và kiểm tra bằng Valgrind

Thao tác	List	Mảng
Chèn đầu	$O(1)$	$O(n)$
Truy cập phần tử i	$O(n)$	$O(1)$
Tìm kiếm	$O(n)$	$O(n)$
Xoá theo giá trị	$O(n)$	$O(n)$
In / giải phóng	$O(n)$	$O(n)$ / $O(1)$

List không có truy cập ngẫu nhiên: không có `list[3]`, cũng không tìm nhị phân được.

Quên gọi `free_list` với danh sách 4 nút:

```
==4321== LEAK SUMMARY:
==4321==    definitely lost: 16 bytes in 1 blocks
==4321==    indirectly lost: 48 bytes in 3 blocks
```

- **definitely**: nút đầu – không còn con trỏ nào trỏ tới.
- **indirectly**: 3 nút còn lại – chỉ được trỏ tới từ nút đã mất.
- Sửa *definitely* trước; *indirectly* thường tự hết.

Nội dung

1 Mở đầu: từ Lab 2 sang Lab 3

2 Hàm

3 Call stack và đệ quy

4 Con trỏ

5 Con trỏ, mảng, chuỗi và tìm kiếm nhị phân

6 Cấp phát bộ nhớ động

7 Lỗi bộ nhớ và Valgrind

8 Danh sách liên kết đơn

9 Bài tập thực hành

10 Tổng kết

Kế hoạch 90 phút và các điểm dừng

Thời lượng	**Nội dung hướng dẫn**	**Sinh viên làm (điểm dừng)**
0–8’	Mục tiêu, bố cục bộ nhớ tiến trình	Chạy `addr.c`
8–20’	Hàm: prototype, tham trị, phạm vi, thiết kế	**Bài 1** chạy được
20–35’	Call stack, gdb `bt`, đệ quy, Fibonacci	**Bài 6, 7**
35–55’	Con trỏ, mảng, chuỗi, tìm kiếm nhị phân	**Bài 11, 12**
55–70’	Cấp phát động, `realloc`	**Bài 16**
70–82’	Lỗi bộ nhớ, Valgrind	`valgrind` cho **Bài 16**
82–88’	Danh sách liên kết đơn (trình diễn)	Đọc và chạy `list.c`
88–90’	Tổng kết, yêu cầu nộp bài	–
Sau 90’	Sinh viên **tự làm** các bài còn lại; giảng viên hỗ trợ tại chỗ	

Quy tắc của buổi thực hành

Gõ tay, không copy–paste từ PDF. Mỗi bài: `gcc -Wall -Wextra -std=c11 -g` **sạch cảnh báo**; chạy ví dụ của đề *và* ít nhất một trường hợp biên; Bài 16–20 thêm một lần chạy `valgrind --leak-check=full`.

Bài 1 – Máy tính tiền điện (Hàm)

```c
#define LIMIT1 50
#define LIMIT2 100
#define PRICE1 1800
#define PRICE2 2300
#define PRICE3 2800

/* Luy tien: moi muc gia chi ap cho
   phan kWh nam trong bac do */
long long electricity_cost(int kwh) {
    long long cost = 0;
    if (kwh > LIMIT2) {
        cost += (long long)(kwh - LIMIT2) * PRICE3;
        kwh = LIMIT2;
    }
    if (kwh > LIMIT1) {
        cost += (long long)(kwh - LIMIT1) * PRICE2;
        kwh = LIMIT1;
    }
    return cost + (long long)kwh * PRICE1;
}
```

```c
void print_bill(int kwh, long long cost) {
    long long vat = cost * 10 / 100;
    printf("So kWh : %d\n", kwh);
    printf("Tien   : %lld\n", cost);
    printf("VAT 10%%: %lld\n", vat);
    printf("Tong   : %lld\n", cost + vat);
}
```

kWh	tiền điện	sau VAT
0	0	0
50	90 000	99 000
51	92 300	101 530
100	205 000	225 500
120	261 000	287 100

Bài 1 – lũy tiến hay một mức giá? Đề chưa nói rõ

Cách hiểu A – lũy tiến (lời giải chọn)

120 kWh = 50 kWh giá bậc 1 + 50 kWh giá bậc 2 + 20 kWh giá bậc 3
$= 90\,000 + 115\,000 + 56\,000 =$ **261 000** đồng.
Đây là cách tính tiền điện sinh hoạt thực tế.

Cách hiểu B – một mức cho toàn bộ

120 kWh thuộc bậc "trên 100" $\Rightarrow$
$120 \times 2\,800 =$ **336 000** đồng.
Chỉ cần sửa thân `electricity_cost`; `main` và `print_bill` giữ nguyên.

Điều được chấm là việc *nêu giả định*

Bài làm tốt ghi rõ cách hiểu trong chú thích đầu file và có bảng kiểm thử ở các biên 0, 50, 51, 100, 101. Nhờ tách hàm, đổi cách hiểu chỉ sửa *một* hàm – đúng lợi ích "sửa một chỗ" ở đầu buổi.

Bẫy: `kwh * 2800` với `int` vẫn an toàn ở đây, nhưng thói quen đúng là tính tiền bằng `long long` – tiền không bao giờ dùng `float`.

Bài 2 – Kiểm tra ngày hợp lệ (Hàm)

```
bool is_leap(int year) {
    return (year % 400 == 0) ||
           (year % 4 == 0 && year % 100 != 0);
}

/* So ngay cua thang; 0 neu thang sai */
int days_in_month(int month, int year) {
    switch (month) {
    case 1: case 3: case 5: case 7:
    case 8: case 10: case 12: return 31;
    case 4: case 6: case 9: case 11: return 30;
    case 2: return is_leap(year) ? 29 : 28;
    default: return 0;
    }
}

bool is_valid_date(int d, int m, int y) {
    if (y < 1) return false;
    return d >= 1 && d <= days_in_month(m, y);
}
```

Nhập	Kết quả
29 2 2024	hợp lệ
29 2 2023	không hợp lệ
31 4 2025	không hợp lệ
29 2 1900	không (chia hết 100)
29 2 2000	hợp lệ (chia hết 400)
0 1 2025, 31 13 2025	không hợp lệ

- Tháng sai $\Rightarrow$ `days_in_month` trả 0 $\Rightarrow$ không cần kiểm tra tháng riêng.
- `bool`: `#include <stdbool.h>`.
- 1900 và 2000 phân biệt bài đúng với bài "gần đúng".

Bài 3 – Máy bán nước tự động (Hàm)

```c
#define TX_OK          0
#define TX_NOT_ENOUGH  1
#define TX_BAD_CODE    2

int price_of(int code) {     /* -1: ma sai */
    switch (code) {
    case 1: return 10000;
    case 2: return 12000;
    case 3: return 15000;
    default: return -1;
    }
}

int check_transaction(int money, int code) {
    int price = price_of(code);
    if (price < 0) return TX_BAD_CODE;
    return money >= price ? TX_OK : TX_NOT_ENOUGH;
}
```

```c
int price = price_of(code);
switch (check_transaction(money, code)) {
case TX_OK:
    printf("Mua %s. Tien thua: %d\n",
           name_of(code), money - price);
    break;
case TX_NOT_ENOUGH:
    printf("Thieu %d dong\n", price - money);
    break;
default:
    printf("Ma san pham khong hop le\n");
}
```

15000 2 → trà chanh, thừa 3000; 8000 1 → thiếu 2000; 20000 4 → mã sai; 12000 2 → thừa 0. `name_of` trả `const char *` trỏ vào hằng chuỗi.

Bài 4 – Thống kê điểm 3 môn (Hàm)

```c
#define EPS 1e-9

double average3(double a, double b, double c) {
    return (a + b + c) / 3.0;
}

double max3(double a, double b, double c) {
    double m = a;
    if (b > m) m = b;
    if (c > m) m = c;
    return m;
}

const char *classify(double avg) {
    if (avg >= 8.0 - EPS) return "Gioi";
    if (avg >= 6.5 - EPS) return "Kha";
    if (avg >= 5.0 - EPS) return "Dat";
    return "Truot";
}
```

- Ngưỡng xét từ **cao xuống thấp** – giống Bài 4 của Lab 2.
- Mỗi hàm trả về một giá trị, `main` quyết định in thế nào.
- `scanf("%lf", &s[i])` cho `double`; kiểm tra $0 \le s \le 10$.

Kiểm chứng: `7.5 8.0 6.5` → `7.33`, cao nhất `8.0`, `Kha`; `6.5 6.5 6.5` → `Kha`; `4.9 5 5` → `Truot`.

Vì sao có EPS?

Xem slide sau – một ca mà in ra `5.00` nhưng lại xếp `Truot`.

Bài 4 – bẫy số thực: in 5.00 nhưng xếp loại Truot

```
$ ./bai04_khong_eps            # classify() so sanh truc tiep avg >= 5.0
Diem mon 1: 2.8
Diem mon 2: 6.1
Diem mon 3: 6.1
Diem trung binh: 5.00
Xep loai       : Truot          # ???
```

- 2.8, 6.1 không biểu diễn chính xác được ở hệ nhị phân.
- Tổng chia 3 ra `4.9999999999999991`: `%.2f` làm tròn thành `5.00`, còn phép so sánh thấy < 5.0.
- Thứ tự cộng cũng ảnh hưởng: `6.1 6.1 2.8` có thể cho kết quả khác.

Hai cách chữa

(1) So sánh với dung sai: `avg >= 5.0 - 1e-9`.
(2) Làm tròn trước khi so: `round(avg * 100) / 100` (cần `-lm`).

Bài học chung: không bao giờ so sánh == hay ngưỡng "sát nút" với `double` mà không nghĩ tới sai số.

Bài 5 – Hoá đơn mua hàng (Hàm)

```
long long subtotal(int qty, long long price) {
    return (long long)qty * price;
}

/* percent: so nguyen 0..100 */
long long discount_amount(long long total,
                          int percent) {
    return total * percent / 100;
}

long long amount_due(long long total,
                     long long discount) {
    return total - discount;
}
```

```
$ echo "3 200000 10" | ./bai05
---------- HOA DON ----------
So luong          : 3
Don gia           : 200000
Tong truoc giam   : 600000
Giam gia ( 10%)   : 60000
So tien phai tra  : 540000
```

- **Giả định**: một loại hàng, tổng = số lượng × đơn giá – theo ví dụ của đề.
- Tiền là số nguyên đồng; % giảm là số nguyên. Giảm 15% của 99 999 ra 14 999 (làm tròn xuống) – nêu rõ trong hoá đơn.
- In % bằng %% trong `printf`.

Bài 6 – Tổng 1 đến n bằng đệ quy

```
long long sum_to(int n) {
    if (n == 1)                 /* S(1) = 1 */
        return 1;
    return n + sum_to(n - 1); /* S(n) */
}
```

Kiểm chứng bằng công thức: $S(n) = n(n+1)/2$.
$5 \to 15$, $1 \to 1$, $100 \to 5050$.

- `main` phải chặn $n \leq 0$: với `n = 0`, điểm dừng `n == 1` *không bao giờ tới* $\Rightarrow$ stack overflow.
- Thử $n = 10^6$ khi biên dịch `-O0`: segfault – độ sâu một triệu tầng vượt 8 MB stack.
- Với `-O2`, gcc có thể tự biến đệ quy này thành vòng lặp nên không crash – **đừng dựa vào điều đó**.
- Kết quả `long long`: $n(n+1)/2$ vượt `int` khi $n > 65\,535$.

Bài 7 – Giai thừa và Bài 8 – Lũy thừa (Đệ quy)

```
#define MAX_N 20        /* 20! vua 64 bit */

unsigned long long factorial(int n) {
    if (n == 0)                 /* 0! = 1 */
        return 1;
    return n * factorial(n - 1); /* n doi
                    sang unsigned long long */
}
```

0 → 1; 1 → 1; 5 → 120; 7 → 5040; 20 → 2432902008176640000; 21 → báo lỗi miền nhập. In bằng `%llu`.

```
long long power(long long a, int n) {
    if (n == 0)          /* a^0 = 1 */
        return 1;
    return a * power(a, n - 1);
}
```

2 5 → 32; 3 4 → 81; -2 3 → (-2)^3 = -8; 5 0 → 1; 0 0 → 1 (quy ước).

- $n < 0$: chặn ở `main` (đề cho $n \geq 0$).
- 2 63 tràn `long long` – UB, `-fsanitize=undefined` báo.
- Mở rộng: $a^n = (a^{n/2})^2$ chỉ cần $O(\log n)$ lần gọi.

Bài 9 – Tổng chữ số và Bài 10 – In ngược (Đệ quy)

```c
int digit_sum(int n) {
    if (n < 10)              /* 1 chu so */
        return n;
    return n % 10 + digit_sum(n / 10);
}
```

1234 → 10; 908 → 17; 7 → 7; 1000000 → 1; 2147483647 → 46.

```c
void print_reversed(int n) {
    printf("%d", n % 10);    /* in TRUOC */
    if (n >= 10)
        print_reversed(n / 10);  /* goi SAU
    */
}
```

12345 → 54321; 7 → 7; 1200 → 0021.

Đổi chỗ hai dòng trong `print_reversed` thì sao?

Gọi đệ quy *trước* rồi mới `printf` ⇒ in *xuôi* 12345: lệnh in chạy khi các frame được *pop* (đi lên). Vị trí lệnh so với lời gọi đệ quy quyết định thứ tự – dò vết bằng gdb để tự thấy.

Nối Lab 2, Bài 6: bản vòng lặp dựng lại số nên 1200 → 21; bản này in từng chữ số nên giữ được 0021 – đề Bài 10 chỉ yêu cầu *in*.

Bài 11 – Hoán đổi và Bài 12 – Tổng, hiệu (Con trỏ)

```c
void swap(int *a, int *b) {
    int tmp = *a;
    *a = *b;
    *b = tmp;
}
/* main */
printf("Truoc: a = %d, b = %d\n", a, b);
swap(&a, &b);
printf("Sau  : a = %d, b = %d\n", a, b);
```

5 9 → 9 5; 7 7 → 7 7. `swap(&a, &a)` vẫn đúng nhờ biến tạm (mẹo XOR thì sai ở ca này).

```c
void sum_diff(int a, int b, int *sum, int *diff) {
    *sum  = a + b;
    *diff = a - b;
}
/* main */
int s, d;
sum_diff(a, b, &s, &d);
printf("Tong = %d, hieu = %d\n", s, d);
```

10 4 → 14, 6; 4 10 → 14, −6.

Lỗi hay gặp

Gọi `swap(a, b)` (thiếu &): gcc báo `passing argument 1 of 'swap' makes pointer from integer`; chạy thì ghi vào địa chỉ "5" ⇒ segfault. **Đọc cảnh báo**, đừng bỏ qua.

Bài 13 – Tìm số lớn nhất trong mảng (Con trỏ)

```c
#define MAX_N 1000

/* n >= 1; const: ham hua khong sua mang */
int max_of(const int *a, int n) {
    int max = a[0];          /* KHONG phai 0 */
    for (const int *p = a + 1; p < a + n; p++)
        if (*p > max)
            max = *p;
    return max;
}

int main(void) {
    int a[MAX_N], n;
    /* doc n trong [1, MAX_N], roi n phan tu */
    printf("Gia tri lon nhat: %d\n", max_of(a, n));
}
```

Kiểm chứng: `[3, 8, 2, 10, 5]` $\to 10$; `[-5, -2, -9]` $\to -2$; `[42]` $\to 42$; $n = 0 \to$ báo lỗi.

- Khởi tạo `max = 0` sai với mảng toàn số âm (cùng bẫy với Bài 10 Lab 2).
- Duyệt bằng con trỏ: `p` chạy từ `a + 1` tới *trước* `a + n`.
- Mảng tĩnh `MAX_N` đặt trần cho n – Bài 16 sẽ bỏ trần này bằng `malloc`.

Bài 14 – Cập nhật số dư tài khoản (Con trỏ)

```
bool deposit(long long *balance,
             long long amount) {
    if (amount <= 0)
        return false;
    *balance += amount;
    return true;
}

bool withdraw(long long *balance,
              long long amount) {
    if (amount <= 0 || amount > *balance)
        return false;
    *balance -= amount;
    return true;
}
```

```
So du ban dau: 300000
Lenh: n <tien> = nap, r <tien> = rut, q = thoat
> r 400000
Giao dich khong hop le. So du: 300000
> n 0
Giao dich khong hop le. So du: 300000
> r 300000
Thanh cong. So du: 0
> q
```

- Hàm *trả trạng thái, sửa số dư qua con trỏ* – mẫu của system call.
- Kiểm tra `amount` **trước** khi sửa: giao dịch lỗi không làm đổi số dư.
- Rút đúng bằng số dư (về 0) là hợp lệ – ca biên nên thử.

Bài 15 – Thống kê mảng bằng con trỏ

```
double total(const double *a, int n) {
    double s = 0;
    for (const double *p = a; p < a + n; p++)
        s += *p;
    return s;
}

int count_pass(const double *a, int n) {
    int c = 0;
    for (const double *p = a; p < a + n; p++)
        if (*p >= 5.0) c++;
    return c;
}

/* MOT ham, HAI ket qua (n >= 1) */
void min_max(const double *a, int n,
             double *min, double *max) {
    *min = *max = a[0];
    for (const double *p = a + 1; p < a + n; p++) {
        if (*p < *min) *min = *p;
        if (*p > *max) *max = *p;
    }
}
```

```
So sinh vien: 4
4.5 7.0 8.5 5.0
Tong diem       : 25.00
So sinh vien dat: 3
Diem cao nhat   : 8.5
Diem thap nhat  : 4.5
```

- "Bằng con trỏ" thể hiện ở hai chỗ: *duyệt* bằng p++ và *trả nhiều kết quả* qua min, max.
- 5.0 là *đạt* ("từ 5 trở lên") – dùng >=.
- $n = 1$: min = max = phần tử duy nhất.

Bài 16 – Điểm với kích thước do người dùng chọn (Cấp phát động)

```c
int n;
if (scanf("%d", &n) != 1 || n <= 0) {
    fprintf(stderr, "Loi: n phai la so nguyen duong\n");
    return 1;
}
double *scores = malloc((size_t)n * sizeof *scores);
if (scores == NULL) {
    perror("malloc");
    return 1;
}
for (int i = 0; i < n; i++) {
    if (scanf("%lf", &scores[i]) != 1
        || scores[i] < 0 || scores[i] > 10) {
        fprintf(stderr, "Loi: diem khong hop le\n");
        free(scores);          /* nhanh loi CUNG free */
        return 1;
    }
}
double sum = 0;
for (int i = 0; i < n; i++)
    sum += scores[i];
printf("Diem trung binh ca lop: %.2f\n", sum / n);
free(scores);
scores = NULL;
```

```
$ gcc -g -Wall bai16.c -o bai16
$ echo "5 6 7 8 9 10" | valgrind \
      --leak-check=full ./bai16
...
Diem trung binh ca lop: 8.00
==4321== HEAP SUMMARY:
==4321==     in use at exit: 0 bytes in 0 blocks
==4321== All heap blocks were freed --
          no leaks are possible
==4321== ERROR SUMMARY: 0 errors from 0 contexts
```

- $n = 0$ bị chặn trước `malloc` – tránh cả `malloc(0)` lẫn chia cho 0.
- Thử xoá dòng `free(scores)` trong nhánh lỗi rồi nhập điểm 11: Valgrind báo `definitely lost`.

Bài 17 – Danh sách chi tiêu (Cấp phát động)

```c
long long sum_of(const long long *a, int n);
long long max_of(const long long *a, int n);
void print_list(const long long *a, int n);

/* main, sau khi doc va kiem tra n > 0: */
long long *spend = malloc((size_t)n * sizeof *spend);
if (spend == NULL) {
    perror("malloc");
    return 1;
}
for (int i = 0; i < n; i++) {
    if (scanf("%lld", &spend[i]) != 1 || spend[i] < 0) {
        fprintf(stderr, "Loi: khoan chi %d\n", i + 1);
        free(spend);
        return 1;
    }
}
print_list(spend, n);
printf("Tong chi           : %lld\n", sum_of(spend, n));
printf("Khoan chi lon nhat : %lld\n", max_of(spend, n));
free(spend);
```

```
So khoan chi: 3
25000 50000 120000
Danh sach da nhap:
  Khoan 1: 25000
  Khoan 2: 50000
  Khoan 3: 120000
Tong chi           : 195000
Khoan chi lon nhat : 120000
```

Cấu trúc giống hệt Bài 16 – điểm mới là các hàm nhận *mảng động* qua `const long long *` y như nhận mảng tĩnh: hàm không cần biết vùng nhớ nằm ở stack hay heap.

Bài 18 – Mở rộng danh sách nhiệt độ (realloc)

```c
double *t = malloc((size_t)n * sizeof *t);
if (t == NULL) { perror("malloc"); return 1; }
if (read_values(t, n) != 0) { free(t); return 1; }

if (scanf("%d", &m) != 1 || m < 0) { free(t); return 1; }
double *tmp = realloc(t, (size_t)(n + m) * sizeof *t);
if (tmp == NULL) {
    perror("realloc");
    free(t);                    /* vung cu van con */
    return 1;
}
t = tmp;

if (read_values(t + n, m) != 0) { free(t); return 1; }
for (int i = 0; i < n + m; i++)
    printf("%s %g", i ? "," : "", t[i]);
free(t);
```

```
So gia tri ban dau n: 3
Nhap 3 nhiet do: 30 31 29
So gia tri them m: 2
Nhap 2 nhiet do moi: 32 30
Danh sach (5 gia tri): 30, 31, 29, 32, 30
```

- `read_values(t + n, m)`: số học con trỏ – ghi tiếp từ `t[n]`.
- Nhiệt độ dùng `double`, in bằng `%g` (`29.5` vẫn đúng).
- $m = 0$ hợp lệ: `realloc` giữ nguyên kích thước.

Bài 19 – Gộp hai danh sách (Cấp phát động)

```c
/* Cap phat va doc n so. NULL neu loi.
   Nguoi GOI so huu vung nho va phai free(). */
int *read_list(int n) {
    int *a = malloc((size_t)n * sizeof *a);
    if (a == NULL) return NULL;
    for (int i = 0; i < n; i++)
        if (scanf("%d", &a[i]) != 1) {
            free(a);          /* don dep truoc khi bao loi */
            return NULL;
        }
    return a;
}

/* main */
int *a = read_list(na);
if (a == NULL) return 1;
int *b = read_list(nb);
if (b == NULL) { free(a); return 1; }
int *c = merge(a, na, b, nb);       /* slide "Ai cap phat" */
if (c == NULL) { free(a); free(b); return 1; }
/* ... in c ... */
free(a); free(b); free(c);
```

```
So phan tu danh sach 1 va 2: 3 2
Nhap 3 so cua danh sach 1: 1 2 3
Nhap 2 so cua danh sach 2: 10 20
Mang gop: 1, 2, 3, 10, 20
```

- Mỗi đường ra giải phóng *đúng* những gì đã cấp phát tới thời điểm đó: 0, 1, 2 rồi 3 vùng.
- Hàm trả vùng `malloc` phải ghi trong chú thích ai `free`.
- Giả định: hai danh sách đều có ít nhất 1 phần tử.

Bài 20 – Hệ thống khảo sát mini (Cấp phát động)

```c
/* Doc 1 danh gia hop le, hoi lai neu sai; -1 neu het du lieu */
int read_rating(int idx) {
    int r;
    for (;;) {
        printf("Danh gia cua nguoi %d (1-5): ", idx);
        int k = scanf("%d", &r);
        if (k == EOF) return -1;
        if (k == 1) {
            if (r >= 1 && r <= 5) return r;
            printf("  -> %d nam ngoai 1..5, nhap lai.\n", r);
        } else {                        /* gap chu, khong phai so */
            if (scanf("%*s") == EOF) return -1;   /* bo tu sai */
            printf("  -> Khong phai so nguyen, nhap lai.\n");
        }
    }
}

int count[6] = {0};           /* count[1..5] */
long sum = 0;
for (int i = 0; i < n; i++) {
    count[ratings[i]]++;      /* gia tri lam CHI SO */
    sum += ratings[i];
}
```

```
Muc 1: 0 lan
Muc 2: 0 lan
Muc 3: 1 lan
Muc 4: 2 lan
Muc 5: 3 lan
Diem trung binh: 4.33
Muc xuat hien nhieu nhat (3 lan): 5
```

- `count[ratings[i]]++` chỉ an toàn vì `read_rating` *bảo đảm* $1 \le r \le 5$.
- Nhiều mức cùng nhiều nhất (1 2 1 2): in *tất cả* – đề chưa nói, nêu giả định.
- Nhập abc không làm treo vòng lặp nhờ `scanf("%*s")`.

Kiểm chứng cả 20 bài bằng một script (kỹ năng Lab 1)

```
#!/bin/bash
# check.sh - bien dich va chay thu moi bai
for f in bai*.c; do
    exe="${f%.c}"
    gcc -Wall -Wextra -std=c11 -g "$f" -o "$exe" \
        || { echo "LOI BIEN DICH: $f"; continue; }
    if [ -f "tests/$exe.in" ]; then
        valgrind -q --leak-check=full --error-exitcode=9 \
            "./$exe" < "tests/$exe.in" > "out/$exe.txt"
        echo "$exe: ma tra ve $?"
    fi
done
```

- `tests/bai16.in` chứa 5 6 7 8 9 10: chạy lại mọi lúc, không gõ tay.
- `--error-exitcode=9`: có lỗi bộ nhớ thì mã trả về là 9 – đọc bằng `$?` như Lab 1.
- `-q`: Valgrind chỉ in khi có lỗi.
- So kết quả với file mong đợi: `diff out/bai16.txt expected/bai16.txt`.

Nội dung

1. Mở đầu: từ Lab 2 sang Lab 3
2. Hàm
3. Call stack và đệ quy
4. Con trỏ
5. Con trỏ, mảng, chuỗi và tìm kiếm nhị phân
6. Cấp phát bộ nhớ động
7. Lỗi bộ nhớ và Valgrind
8. Danh sách liên kết đơn
9. Bài tập thực hành
10. Tổng kết

Câu hỏi kiểm tra trong báo cáo

1. Vì sao `swap(int a, int b)` không đổi được hai biến của `main`?
2. Bỏ prototype của một hàm gọi trước khi định nghĩa thì gcc báo gì, vì sao?
3. Biến `static` cục bộ khác biến cục bộ thường ở điểm nào?
4. Một stack frame chứa gì? Xem call stack bằng lệnh `gdb` nào?
5. Hai điều kiện của một hàm đệ quy đúng? Bài 6 với $n = 0$ thì sao?
6. Vì sao `fib_rec(40)` cần hơn 331 triệu lời gọi?
7. `arr[i]` và `*(arr + i)` liên hệ thế nào? Vì sao `arr++` lỗi còn `p++` thì không?
8. Vì sao `sizeof(arr)` trong hàm cho 8?
9. `char *s = "Hi"` và `char s[] = "Hi"` khác nhau thế nào?
10. Vì sao không viết `p = realloc(p, n)`?
11. `malloc` có phải system call không? Kiểm chứng bằng gì?
12. Valgrind: *definitely lost* khác *indirectly lost* thế nào?

Tiêu chí chấm bài

Đúng đắn – Sạch cảnh báo – **Tổ chức hàm** – Bền vững (kiểm tra nhập và `malloc`, `free` cả ở nhánh lỗi) – **Sạch Valgrind** (Bài 16–20) – Hiểu biết (nêu giả định) – Kiểm chứng tại trường hợp biên.

Chuẩn bị cho tuần sau: Struct, union, enum và sắp xếp

Đọc trước

Tài liệu *Lab 4. Advanced C type and Sorting*, mục 1 (struct, enum, typedef, union) và mục 2 (con trỏ tới cấu trúc); K&R, *The C Programming Language*, Chương 6 (Structures).

Luyện thêm nếu còn thời gian

- Viết lại Bài 6–10 bằng vòng lặp; với Bài 8, viết bản $O(\log n)$: $a^n = (a^{n/2})^2$.
- Chạy `valgrind` cho cả 5 bài cấp phát động; cố ý xoá một `free` để đọc báo cáo.
- Thêm vào `list.c` hàm `count_nodes` và `insert_tail`; kiểm tra bằng Valgrind.
- Viết Bài 19 cách khác: cấp phát mảng 1 rồi `realloc` để nối mảng 2 vào cuối.

Câu hỏi dẫn sang tuần sau

`struct Node` gồm một `int` (4 byte) và một con trỏ (8 byte), nhưng `sizeof` và Valgrind đều báo 16 byte. Bốn byte còn lại ở đâu? → **căn lề và đệm** (alignment & padding) của `struct`.

Cảm ơn đã lắng nghe!